



King: 15, 19 & 17.1..4&7

HE1025 Operativsystem

Ö1: HI1024 The missing chapter...

SVÄRT



Kursen...

F1: V#1 Processorn.

Ö1: V#2 c Pekare...

F2: V#3 Schemaläggning

L1: V#4 FreeRTOS

F3: V#5 Segmentation

Ö2: V#6 Eget arbete

F4: V#7 Paging

L2: V#8 Paging

F5: C#1 Locks & CV

Ö3: C#2 LINUX

F6: C#3 Semaphores

L3: C#4 Barberare

F7: P#1 Storage

Ö4: P#2 File/DMA

F8: P#3 Communication

L4: P#4 Client/Server

Kap (1) 2..4 & 6

Kap 14 & c-boken

Kap 7..9,(10-11), RTOS

Lab #1 Handledning

Kap 12..13, 15..17

-

Kap 18-22, (23-24)

Lab #2 Handledning

Kap (25) 26..27 & 28..29.1

Kap 5 (39)

Kap 30..33, (34)

Lab #3 Handledning

Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)

-

Kap (47, 39) & 48, (49..51)

Lab #4 Handledning

TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

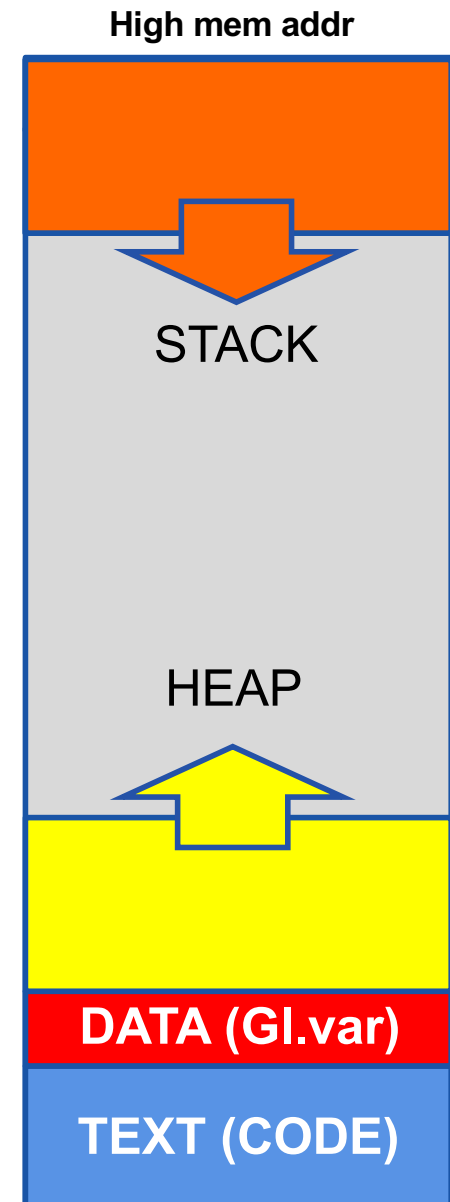
LAB1: Närvaro L1..4

Övning 1: Pekare i C på riktigt!



HI1024
THE MISSING
CHAPTER!

F1: Text, data, stack & heap.





Av tre anledningar...

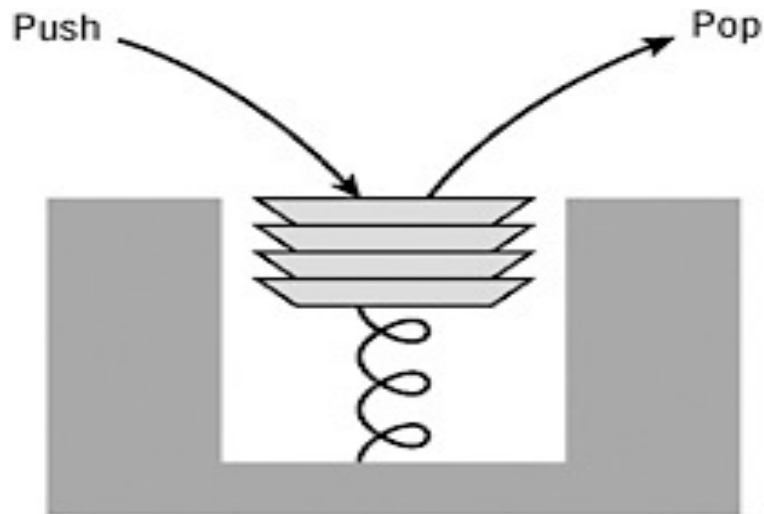
- Vi vill att ni ska utveckla kod enligt objektorienterade principer, även om det språk ni använder inte är objektorienterat. Inkapsling, modularisering och data-abstraktion ska eftersträvas.
- För att över huvud taget kunna anropa api-funktioner som finns via os-api:er krävs mer insikt i c än vi hann med i HI1024.
- Förstår ni det här, om det än är i sommar när det fått sjunka in, så kommer ni lätt att ta till er java-kursen i åk 2.



Vi lär oss genom ett exempel:

Datastrukturen stack...

(En First In Last Out (FILO) kö.)



Stack

(Implementation)

init

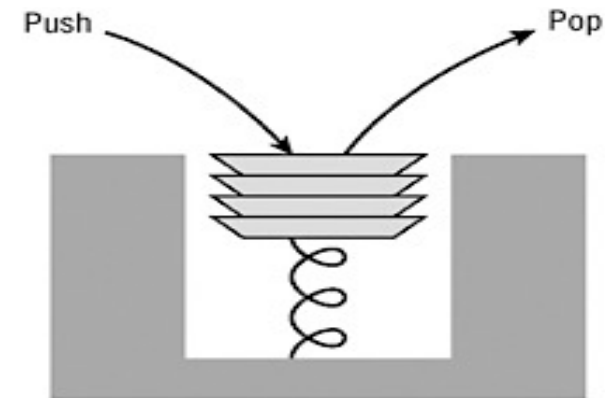
push

pop

Hur?

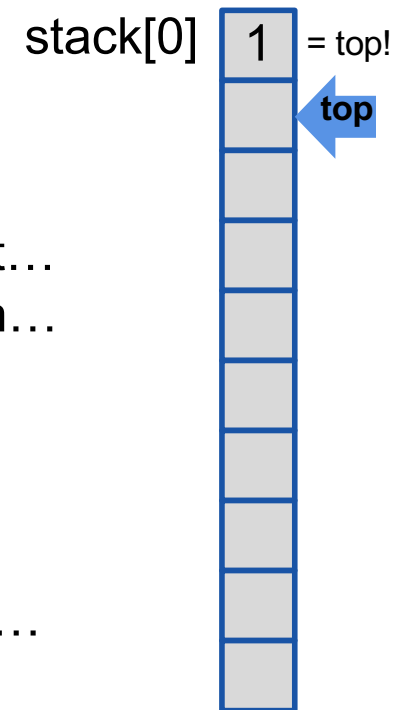
Vi lär oss genom ett exempel:

Datastrukturen stack...



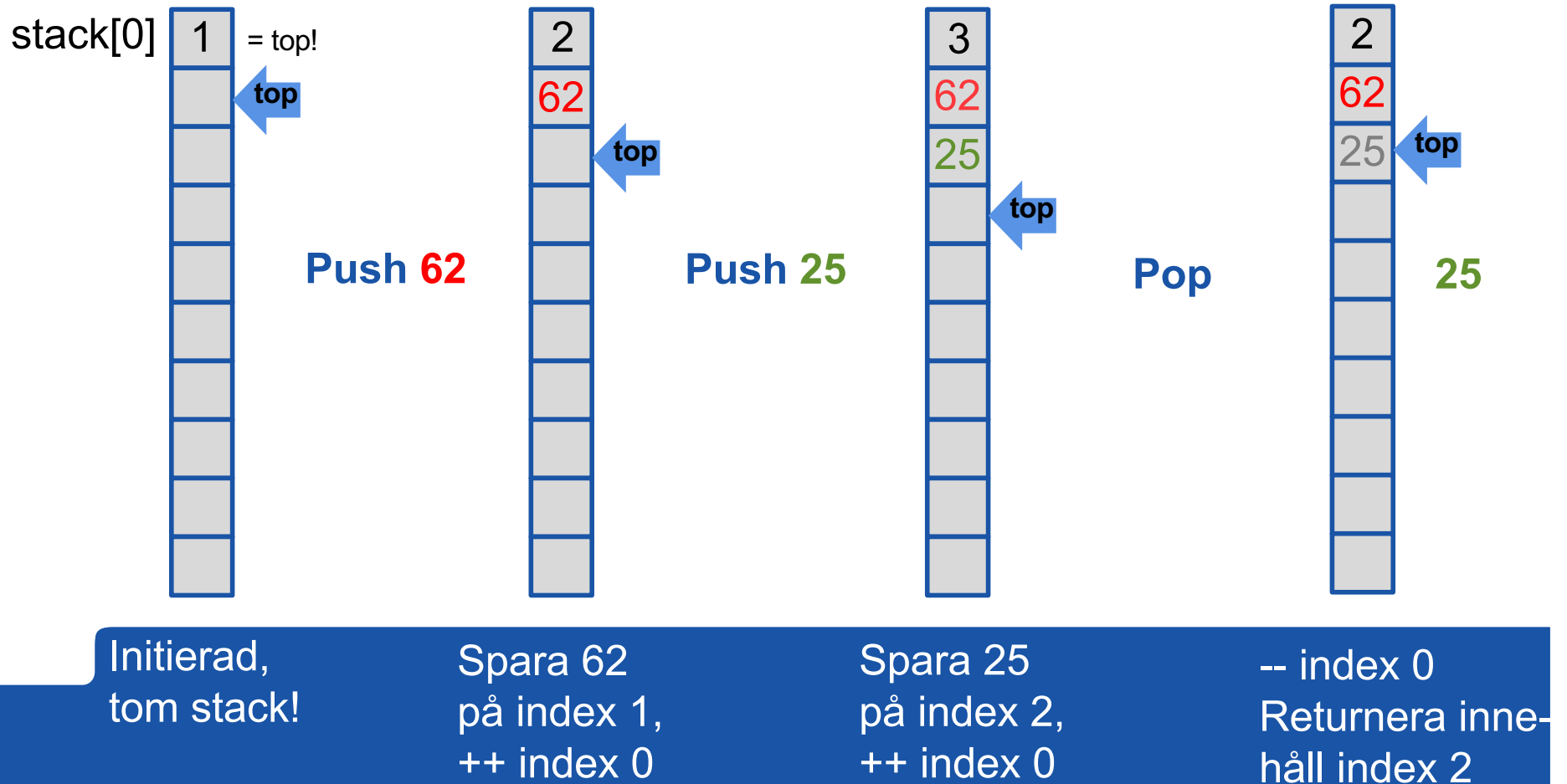
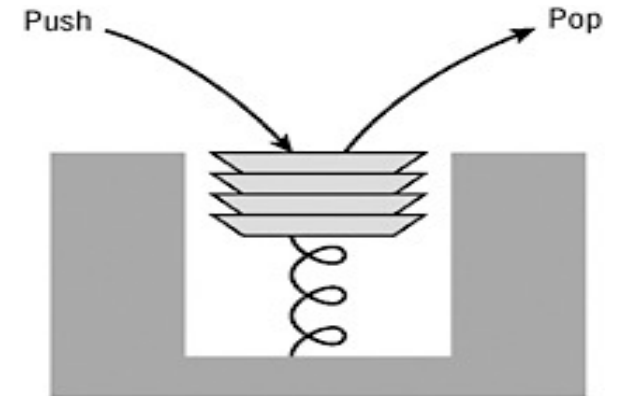
Hum...

- ...en 1-dim matris kan kanske användas...
- ...det är svårt att "trycka ner" tal...
- ...men ingen vet om talet hamnar "sist" i stället...
- ...indexet `top` får markera nästa lediga position...
- ...**push** placerar talet på indexet `top` och ökar `top` med ett...
- ...**pop** returnerar talet på indexet `top-1` och minskar `top` med ett...
- ... varför inte spara indexet `top` först i matrisen...
- ...ok att skippa alla tester för fel.



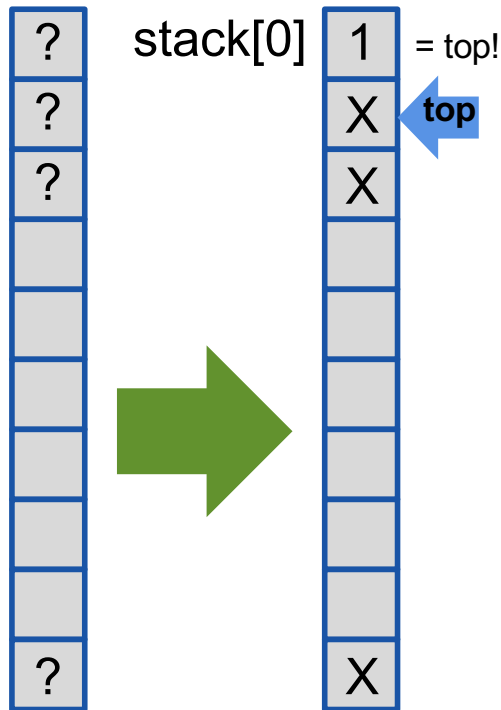
Vi kollar om det fungerar...

Datastrukturen stack...





Stack paketet...



Initierad,
tom stack!

```
#include <stdio.h>

/** Package (int) Stack (No safety!) ***** 1.0 AC *****
void initStack(int aStack[]){           // Constructor
```

```
int main(int argc, char **argv)
{
    int theStack[10];           // Stack impl. visible!
    printf("Stack 1.0 AC\n");

    initStack(theStack);

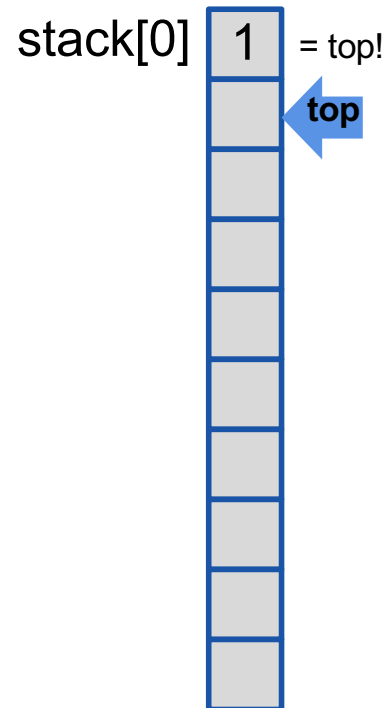
    pushStack(theStack,62);
    pushStack(theStack,25);

    while (!isEmpty(theStack))
        printf("[%d]\n",popStack(theStack));

    printf("Stack EXIT!\n");
    return 0;
}
```




Stack paketet...



Initierad,
tom stack!

```
#include <stdio.h>

/** Package (int) Stack (No safety!) ***** 1.0 AC *****
void initStack(int aStack[]){           // Constructor
    aStack[0]=1;
}
```

```
int main(int argc, char **argv)
{
    int theStack[10];           // Stack impl. visible!
    printf("Stack 1.0 AC\n");

    initStack(theStack);

    pushStack(theStack,62);
    pushStack(theStack,25);

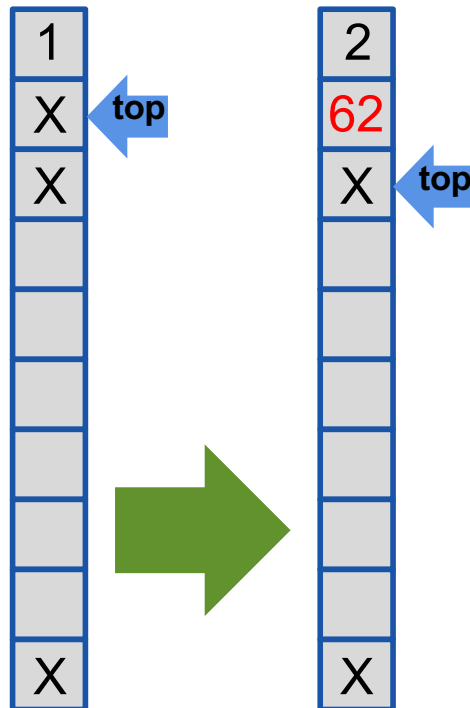
    while (!isEmpty(theStack))
        printf("[%d]\n",popStack(theStack));

    printf("Stack EXIT!\n");
    return 0;
}
```

Stack 1.0 AC
[25]
[62]
Stack EXIT!



Stack paketet...



Spara 62
på index 1,
++ index 0

```
#include <stdio.h>

/** Package (int) Stack (No safety!) ***** 1.0 AC *****
void initStack(int aStack[]){           // Constructor
    aStack[0]=1;
}
void pushStack(int aStack[], int aValue){ // Method
```

```
int main(int argc, char **argv)
{
    int theStack[10];           // Stack impl. visible!
    printf("Stack 1.0 AC\n");

    initStack(theStack);

    pushStack(theStack,62);
    pushStack(theStack,25);

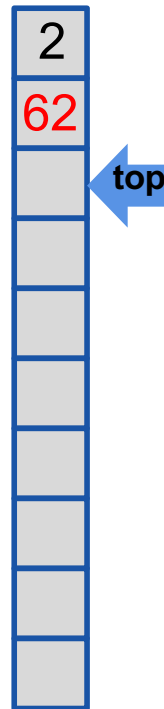
    while (!isEmpty(theStack))
        printf("[%d]\n",popStack(theStack));

    printf("Stack EXIT!\n");
    return 0;
}
```

```
Stack 1.0 AC
[25]
[62]
Stack EXIT!
```



Stack paketet...



Spara 62
på index 1,
++ index 0

```
#include <stdio.h>

/** Package (int) Stack (No safety!) ***** 1.0 AC *****
void initStack(int aStack[]){           // Constructor
    aStack[0]=1;
}
void pushStack(int aStack[], int aValue){ // Method
    aStack[aStack[0]++]=aValue;
}
```

```
int main(int argc, char **argv)
{
    int theStack[10];           // Stack impl. visible!
    printf("Stack 1.0 AC\n");

    initStack(theStack);

    pushStack(theStack,62);
    pushStack(theStack,25);

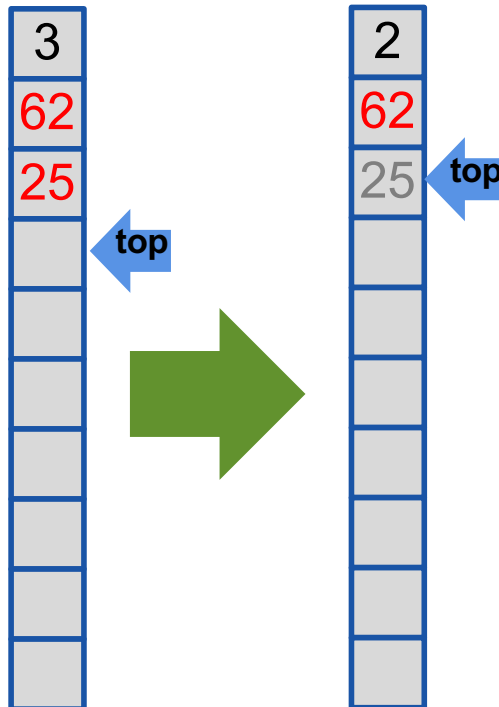
    while (!isEmpty(theStack))
        printf("[%d]\n",popStack(theStack));

    printf("Stack EXIT!\n");
    return 0;
}
```

```
Stack 1.0 AC
[25]
[62]
Stack EXIT!
```



Stack paketet...



-- index 0
Returnera
index 2

```
#include <stdio.h>

/** Package (int) Stack (No safety!) ***** 1.0 AC *****
void initStack(int aStack[]){           // Constructor
    aStack[0]=1;
}
void pushStack(int aStack[], int aValue){ // Method
    aStack[aStack[0]++]=aValue;
}
int popStack(int aStack[]){             // Method
```

```
int main(int argc, char **argv)
{
    int theStack[10];           // Stack impl. visible!
    printf("Stack 1.0 AC\n");

    initStack(theStack);

    pushStack(theStack,62);
    pushStack(theStack,25);

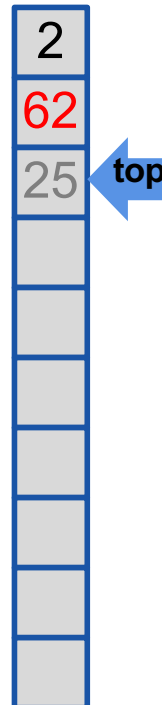
    while (!isEmpty(theStack))
        printf("[%d]\n",popStack(theStack));

    printf("Stack EXIT!\n");
    return 0;
}
```

```
Stack 1.0 AC
[25]
[62]
Stack EXIT!
```



Stack paketet...



-- index 0
Returnera
index 2

```
#include <stdio.h>

/** Package (int) Stack (No safety!) ***** 1.0 AC *****
void initStack(int aStack[]){           // Constructor
    aStack[0]=1;
}

void pushStack(int aStack[], int aValue){ // Method
    aStack[aStack[0]++]=aValue;
}

int popStack(int aStack[]){             // Method
    return aStack[--aStack[0]];
}

int isEmpty(int aStack[]){              // Method
    return (aStack[0]==1);
}

int main(int argc, char **argv)
{
    int theStack[10];                   // Stack impl. visible!

    initStack(theStack);

    pushStack(theStack,62);
    pushStack(theStack,25);

    while (!isEmpty(theStack))
        printf("[%d]\n",popStack(theStack));

    printf("Stack EXIT!\n");
    return 0;
}
```

```
Stack 1.0 AC
[25]
[62]
Stack EXIT!
```




.h + .c + #include

Vi använder det vi lärt oss och flyttar deklARATIONERNA till en .h fil som blir vårt **gränssnitt** (interface) mot de **tjänster** (services) som finns definierade i motsvarande .c-fil.

Vi kan använda tjänsterna, via gränssnittet utan att vi vet exakt hur de är implementerade!

Abstraction,
Reusability &
Maintainability!

```
main.c  intArrStack.h X  intArrStack.c
1  //*** Package (int) Stack (No safety!) ***** 2.0 AC *****
2  void initStack(int aStack[]); // Constructor
3  void pushStack(int aStack[], int aValue); // Method
4  int popStack(int aStack[]); // Method
5  int isEmpty(int aStack[]); // Method
```

```
main.c  intArrStack.h  intArrStack.c X
1  #include "intArrStack.h"
2  //*** Package (int) Stack (No safety!) ***** 2.0 AC *****
3  void initStack(int aStack[]){ // Constructor
4      aStack[0]=1;
5  }
6  void pushStack(int aStack[], int aValue){ // Method
7      aStack[aStack[0]++]=aValue;
8  }
9  int popStack(int aStack[]){ // Method
10     return aStack[--aStack[0]];
11 }
12 int isEmpty(int aStack[]){ // Method
13     return (aStack[0]==1);
14 }
```

```
main.c X  intArrStack.h  intArrStack.c
1  #include <stdio.h>
2  #include "intArrStack.h"
3  int main(int argc, char **argv)
4  {
5      int theStack[10]; // Stack impl. visible!
6      printf("Stack 2.0 AC\n");
7
8      initStack(theStack);
9
10     pushStack(theStack,62);
11     pushStack(theStack,25);
12
13     while (!isEmpty(theStack))
14         printf("[%d]\n",popStack(theStack));
15
16     printf("Stack EXIT!\n");
17     return 0;
18 }
```

Stack 2.0 AC
[25]
[62]
Stack EXIT!



Vad vi strävar efter...

- **High cohesion:** Funktionerna i en modul är starkt relaterade till varandra, bidrar till samma mål. Modulen blir lättare att förstå.
- **Low coupling:** En modul är så fristående som möjligt från andra moduler. Då blir det lättare att återanvända modulen och det blir lättare att underhålla koden.

Det här leder ofta till fyra typer av moduler...

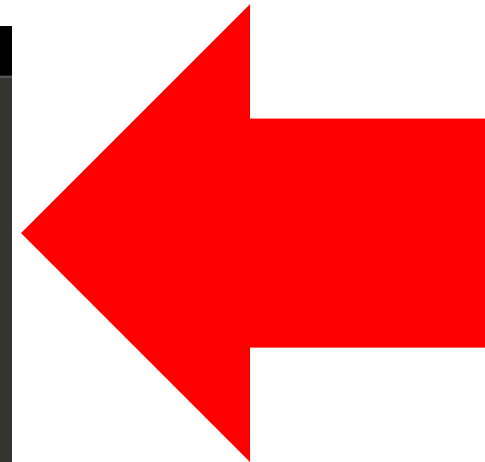
- **Data pool:** En samling konstanter (limits.h)
- **A Library:** En samling relaterade funktioner (string.h)
- **(Abstract) Object:** (osynlig) Data med tillhörande tjänster (stack.h)
- **Abstract Data Type:** En typ för att skapa variabler som kan manipuleras via synliga tjänster, utan insikt om implementationen! (stack++.h?)

Information Hiding: Security (can't tamper) & Flexibility (change imp.)



...huvudproblemet!

```
main.c X  intArrStack.h  intArrStack.c
1  #include <stdio.h>
2  #include "intArrStack.h"
3  int main(int argc, char **argv)
4  {
5      int theStack[10];          // Stack impl. visible!
6      printf("Stack 2.0 AC\n");
7
8      initStack(theStack);
9
10     pushStack(theStack,62);
11     pushStack(theStack,25);
12
13     while (!isEmpty(theStack))
14         printf("[%d]\n",popStack(theStack));
15
16     printf("Stack EXIT!\n");
17     return 0;
18 }
```



...det går inte i c att göra en "privat" Stack-typ, men nästan...



(Vi behöver inte längre tricket med top i Stack[0])

C incomplete types...

```
struct stack {  
    int contents[10];  
    int top;  
};  
typedef struct stack *Stack;
```

Det finns ett trick med poster som vi inte provat...

...skapar vi en typ som är en pekare till en post...

...så behöver inte kompilatorn veta hur posten ser ut!

(Den vet vilken sort posten ska vara (för typkontroll),
och den vet hur stor plats en pekare tar,
så det är först när en variabel av posttypen ska skapas,
som den måste ha passerat den fulla def. av posten!)

Först...

```
struct stack;  
typedef struct stack *Stack;
```



...senare!

```
struct stack {  
    int contents[10];  
    int top;  
};
```



Hum...

Om vi använder den inkompleta specifikationen av en stack-post i .h-filen för att definiera en stack-pekare typ...

...så skulle användaren aldrig "se" hur en stack var definierad...

intADTstack.h

```
/** Package (int) Stack (No safety!) ***** 2.0 AC *****  
struct stack; // Incomp. dec.  
typedef struct stack *Stack;  
Stack newStack(void); // Constructor  
void pushStack(Stack aStack, int aValue); // Method  
int popStack(Stack aStack); // Method  
int isEmpty(Stack aStack); // Method
```

...för det finns bara beskrivet i .c-filen...

intADTstack.c

```
/** Package (int) Stack (No safety!) ***** 3.0 AC *****  
#include <stdlib.h>  
#include "intADTstack.h"  
struct stack {  
    int contents[10];  
    int top;  
};  
Stack newStack(void){ // Constructor
```

...initStack måste också förändras?!

(Dataypen Stack är en pekare till datatypen struct stack... ...en variabel av den sorten borde benämnas t.ex. pStack men då tanken är att användaren ska se variabeln som objektet en stack så utelämnas vanligtvis p:et. A:et kan ses som både "en" stack och "abstract" stack!)



När skapas själva stacken?!

Här reserveras vanligtvis
4*10 byte för variabeln
theStack[] från processens
Stack segment!

```
int theStack[10];           // Stack impl. visible!  
printf("Stack 2.0 AC\n");  
  
initStack(theStack);
```

Här reserveras vanligtvis 4
byte för variabeln theStack
från processens Stack
segment! **De 4 byten är
tänkta att innehålla en
adress till den plats i
minnet där den "riktiga"
stacken finns?!**

```
Stack theStack; // Stack impl. invisible!  
printf("Stack 3.0 AC\n");  
  
theStack = newStack();
```



**newStack måste kunna skapa plats
i minnet, när programmet körs, för
en variabel som är den riktiga
stacken!**

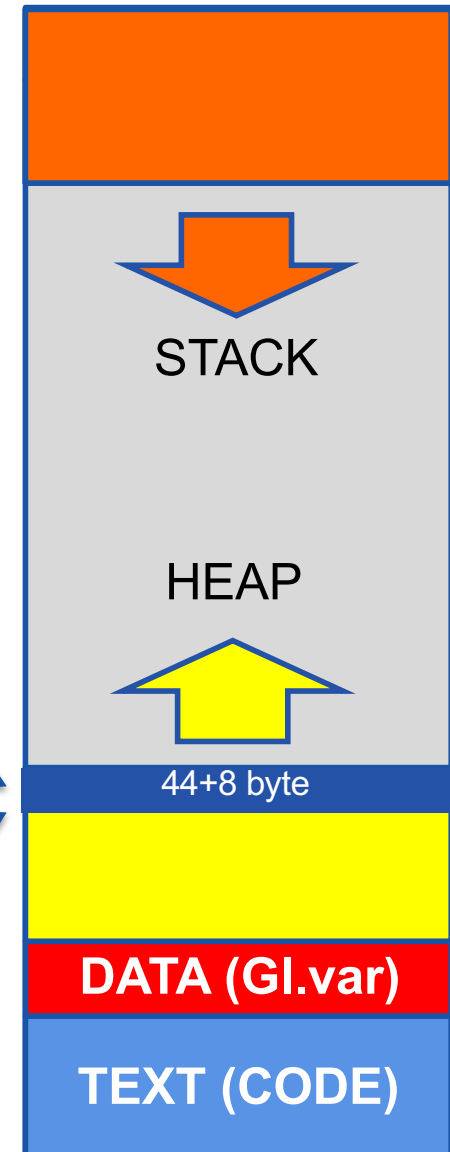
Tursamt nog går det...

```

/** Package (int) Stack (No safety!) ***** 3.0 AC *****
#include <stdlib.h>
#include "intADTstack.h"
struct stack {
    int contents[10];
    int top;
};
Stack newStack(void){ // Constructor
    Stack aStack = malloc(sizeof(struct stack));
    aStack->top=0;
    return aStack;
}
void pushStack(Stack aStack, int aValue){ // Method
    aStack->contents[(aStack->top)++]=aValue;
}
int popStack(Stack aStack){ // Method
    return aStack->contents[--(aStack->top)];
}
int isEmpty(Stack aStack){ // Method
    return (aStack->top==0);
}
void destroyStack(Stack aStack){ // Destructor
    free(aStack);
}

```

High mem addr



malloc() & free()!





Version 3.0...

Nu går det också att lätt lägga till lite felkontroll...

...pushStack, som är void, kan ändras så den returnerar true/false om det gick bra/dåligt...

...och popStack kan returnera en NULL-pointer om stacken var tom...

...vi hoppar över det för att ha en så ren/kort kod som möjligt!

```
#include <stdio.h>
#include "intADTstack.h"
int main(int argc, char **argv)
{
    Stack theStack; // Stack impl. invisible!
    printf("Stack 3.0 AC\n");

    theStack = newStack();

    pushStack(theStack, 62);
    pushStack(theStack, 25);

    while (!isEmpty(theStack))
        printf("[%d]\n", popStack(theStack));

    destroyStack(theStack);

    printf("Stack EXIT!\n");
    return 0;
}
```

```
Stack 3.0 AC
[25]
[62]
Stack EXIT!
```

Då återstår ett problem: Vi kan skapa många stackar, men varje skapad stack kan bara hantera heltal (int)...



void *

Det finns ett till trick i c-språket som vi måste titta på...

Att * i en deklaration betyder "adressen till" vet vi...

Otroligt nog är det ok att skriva **void ***...

Vilket betyder adressen till vad som helst!

När adressens innehåll ska användas måste dock en explicit cast inkluderas för att kompilatorn ska bli nöjd!

```
Test void * 1.0 AC
[42]
Test void done!
```

```
#include <stdio.h>

int main(int argc, char **argv)
{
    int nb = 42;
    void *p = &nb; //Pointer to anything!
    printf("Test void * 1.0 AC\n");

    //Expect p to point to an int!
    printf("[%d]\n", *(int *)p);

    printf("Test void done!\n");
    return 0;
}
```




Version 4.0

Stacken hanterar inte de exakta objekten, utan på stacken finns adressen till de objekt som lagrats där!

WOW!

(Tyvärr så tappar vi all typkontroll!)

```
genADTstack.h X genADTstack.c main.c
1 //*** Package generic Stack (No safety!) ***** 4.0 AC *****
2 struct stack; // Incomp. dec.
3 typedef struct stack *Stack;
4 Stack newStack(void); // Constructor
5 void pushStack(Stack aStack, void *pValue); // Method
6 void *popStack(Stack aStack); // Method
7 int isEmpty(Stack aStack); // Method
8 void destroyStack(Stack aStack); // Destructor
```

```
genADTstack.h genADTstack.c X main.c
1 //*** Package generic Stack (No safety!) ***** 4.0 AC *****
2 #include <stdlib.h>
3 #include "genADTstack.h"
4 struct stack {
5     void *contents[10];
6     int top;
7 };
8 Stack newStack(void){ // Constructor
9     Stack aStack = malloc(sizeof(struct stack));
10    aStack->top=0;
11    return aStack;
12 }
13 void pushStack(Stack aStack, void *pValue){ // Method
14     aStack->contents[(aStack->top)++]=pValue;
15 }
16 void *popStack(Stack aStack){ // Method
17     return aStack->contents[--(aStack->top)];
18 }
19 int isEmpty(Stack aStack){ // Method
20     return (aStack->top==0);
21 }
22 void destroyStack(Stack aStack){ // Destructor
23     free(aStack);
24 }
```




4.0

```
#include <stdio.h>
#include "genADTstack.h"
int main(int argc, char **argv)
{
    Stack theStack; // Stack impl. invisible!
    printf("Stack 4.0 AC\n");

    theStack = newStack();

    pushStack(theStack, "62");
    pushStack(theStack, "25");

    while (!isEmpty(theStack))
        printf("[%s]\n", (char *)popStack(theStack));

    destroyStack(theStack);

    printf("Stack EXIT!\n");
    return 0;
}
```

```
Stack 4.0 AC
[25]
[62]
Stack EXIT!
```

(Stacken kan bara hantera adresser, men det kan lätt hanteras.)



Funktionspekare...

Det finns ett trick kvar som vi måste förstå...

En parameter kan vara en hel funktion! För att sända med en hel funktion som argument anges funktionsnamnet utan parenteser. Argumentets värde blir adressen till den första instruktionen i funktion.

Den formella parameter som tar emot argumentet ska ha samma deklaration som argumentets funktion förutom att **funktionsnamnet ska föregås av en stjärna '*'**.

När funktionen ska "användas" ska funktionsnamnet igen föregås av '*' samt att '* funktionsnamn' ska omslutas av ().



5.0

En "helt vanlig" funktion
som vi vill utföra på
varje objekt som finns
"utpekad" i en stack...

Ny funktion som tar en
stack och en **funktion!**
som argument!!!

(execStack är en
iterator!)

```
genADTstack.h  genADTstack.c  main.c X
1  #include <stdio.h>
2  #include "genADTstack.h"
3
4  void dbg(void *pItem){
5      printf("[%s]\n", (char *)pItem);
6  }
7
8  int main(int argc, char **argv)
9  {
10     Stack theStack; // Stack impl. invisible!
11     printf("Stack 5.0 AC\n");
12
13     theStack = newStack();
14
15     pushStack(theStack, "62");
16     pushStack(theStack, "25");
17
18     execStack(theStack, dbg);
19
20     while (!isEmpty(theStack))
21         printf("[%s]\n", (char *)popStack(theStack));
22
23     destroyStack(theStack);
24
25     printf("Stack EXIT!\n");
26     return 0;
27 }
```



5.0

```
genADTstack.h  genADTstack.c  main.c X
1  #include <stdio.h>
2  #include "genADTstack.h"
3
4  void dbg(void *pItem){
5      printf("[%s]\n", (char *)pItem);
6  }
```

```
genADTstack.h X  genADTstack.c  main.c
1  /*** Package generic Stack (No safety!) ***** 5.0 AC *****
2  struct stack; // Incomp. dec.
3  typedef struct stack *Stack;
4  Stack newStack(void); // Constructor
5  void pushStack(Stack aStack, void *pValue); // Method
6  void *popStack(Stack aStack); // Method
7  int isEmpty(Stack aStack); // Method
8  void execStack(Stack aStack,
9                void (*exec)(void *pItem)); // Iterator
10 void destroyStack(Stack aStack); // Destructor
```

impl. invisible!

```
Stack 5.0 AC
[62]
[25]
[25]
[62]
Stack EXIT!
```

```
18  execStack(theStack, dbg);
```

```
void execStack(Stack aStack, // Iterator
               void (*exec)(void *pItem)){
    for (int i=0; i<aStack->top; i++)
        (*exec)(aStack->contents[i]);
}
```

```
25  printf("Stack EXIT!\n");
26  return 0;
27  }
```



FreeRTOS...

Funktionspekare!

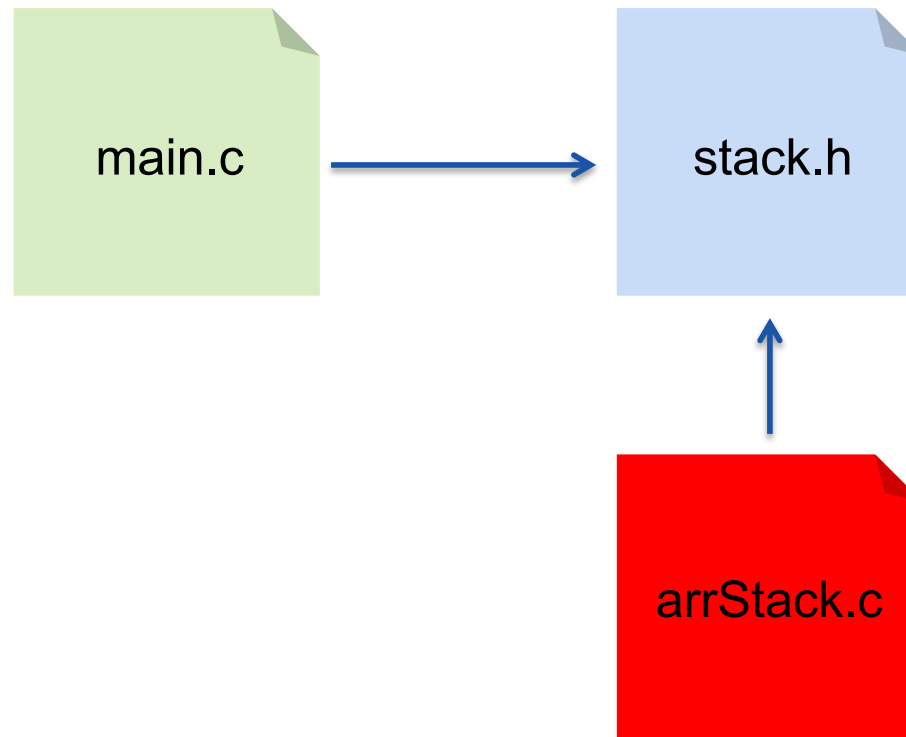
```
BaseType_t xTaskCreate(  
    TaskFunction_t pvTaskCode,  
    const char * const pcName,  
    unsigned short usStackDepth,  
    void *pvParameters,  
    UBaseType_t uxPriority,  
    TaskHandle_t *pxCreatedTask  
);
```

void *

Ni ska kunna skicka en funktion, och relevanta parametrar, till FreeRTOS så den kan starta funktionen som ett eget litet program!

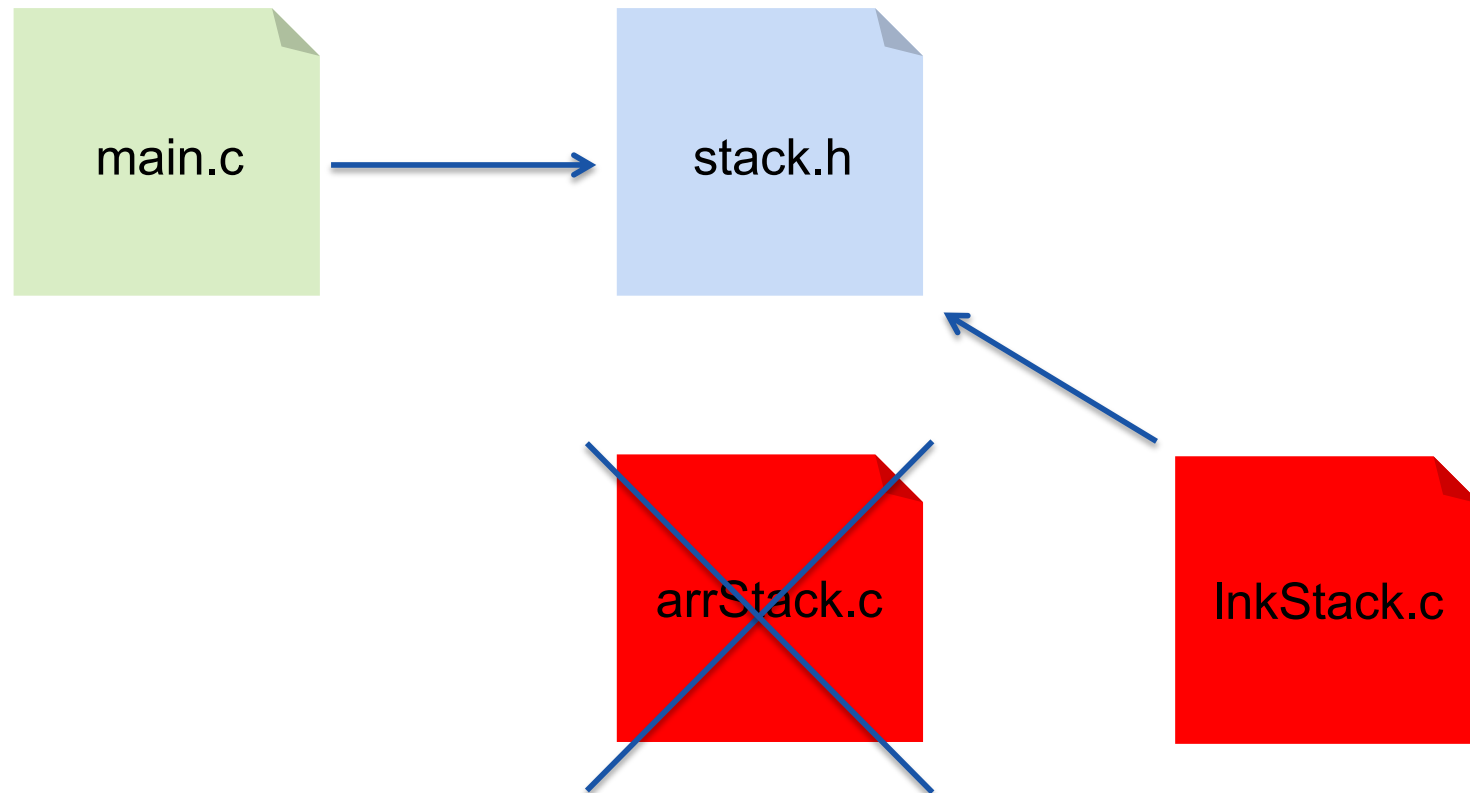


En annan poäng...



Den som använder stack-paketet vet inte att stacken är en array, som kan bli full...

En annan poäng...



...men tack vare uppdelningen i en .h & en .c fil kan vi byta implementeringen till en som använder dynamiskt skapade länkade listor som inte kan ta slut (om inte minnet tar slut) utan att det påverkar main.c! Mer om konceptet i kommande kurser.



Orientering: koppling till c++/java

```
C account.h × C account.c C acctest.c
C account.h > newAccount(char *)
1 struct instVar;
2 typedef struct instVar *Instans;
3 struct Account {
4     void (*deposit)(void *self, int amount);
5     void (*free)(void *self);
6     char *(*toString)(void *self);
7     Instans var;
8 };
9 typedef struct Account Account;
10
11 Account newAccount(char *name);
```

Account

-var	Id Name Amount dbg
------	-----------------------------

+deposit
+free
+toString



Orientering: koppling till c++/java

Account

-var Id
 Name
 Amount
 dbg

+deposit
+free
+toString

```
C account.h  C account.c  C acctest.c  X
C acctest.c > main(void)
1  #include <stdio.h>
2  #include "account.h"
3
4  int main(void){
5      Account acc1 = newAccount("Anders");
6      Account acc2 = newAccount("Oscar");
7      acc2.deposit(&acc2,100);
8
9      printf("%s\n", acc1.toString(&acc1));
10     printf("%s\n", acc2.toString(&acc2));
11
12     acc1.free(&acc1);
13     acc2.free(&acc2);
14 }
```

Account
Anders
0

Account
Oscar
100

```
Anderss-MBP:vsws ac$ gcc -c account.c
Anderss-MBP:vsws ac$ gcc -c acctest.c
Anderss-MBP:vsws ac$ gcc -o accprg acctest.o account.o
Anderss-MBP:vsws ac$ ./accprg
000000:  Anders  +0
000001:  Oscar  +100
Anderss-MBP:vsws ac$
```



Orientering: koppling till c++/java

Account

-var Id
 Name
 Amount
 dbg

+deposit
+free
+toString

```
C account.h   C account.c ×   C acctest.c
C account.c > newAccount(char *)
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "account.h"
5
6  static int accNb=0;
7
8  struct instVar {
9      int id;
10     char name[10];
11     int amount;
12     char dbg[30];
13 };
14
15 void deposit(void *pAccount, int amount){
16     ((Account *)pAccount)->var->amount+=amount;
17 }
18
19 char *toString(void *pAccount){
20     sprintf(((Account *)pAccount)->var->dbg, "%06d:%10s %4d",
21           ((Account *)pAccount)->var->id,
22           ((Account *)pAccount)->var->name,
23           ((Account *)pAccount)->var->amount);
24     return ((Account *)pAccount)->var->dbg;
25 }
```

```
26
27 void garbageCollect(void *pAccount){
28     free(((Account *)pAccount)->var);
29 }
30
31 Account newAccount(char *name){
32     Account self;
33     self.deposit=deposit;
34     self.free=garbageCollect;
35     self.toString=toString;
36     self.var = malloc(sizeof(struct instVar));
37     self.var->id=accNb++;
38     strcpy(self.var->name, name);
39     self.var->amount=0;
40     return self;
41 }
```



OK. Vi övar på det här...

```
#include <stdio.h>
#include "arrCarDB.h"
int main(int argc, char **argv)
{
    CDB myGarage;
    printf("Car DB 1.0 AC\n");

    myGarage = newCDB();
}
```

```
// *** Array based Car DataBase 1.0 AC *****
struct CarDB;
typedef struct CarDB *CDB;
CDB newCDB(void);
```

arrCarDB.h

```
#include <stdlib.h>
#include <string.h>
#include "arrCarDB.h"
typedef struct {
    char id[6+1];
    char type[9+1];
    char color[9+1];
} Car;

struct CarDB {
    Car DB[10];
    int cars;
};

CDB newCDB(void){
    CarDB *CDB = (CarDB *) malloc(sizeof(CarDB));
    CDB->cars = 0;
    return CDB;
}
```

arrCarDB.c



OK. Vi övar på det här...

```
#include <stdio.h>
#include "arrCarDB.h"
int main(int argc, char **argv)
{
    CDB myGarage;
    printf("Car DB 1.0 AC\n");

    myGarage = newCDB();
}
```

```
// *** Array based Car DataBase 1.0 AC *****
struct CarDB;
typedef struct CarDB *CDB;
CDB newCDB(void);
```

arrCarDB.h

```
#include <stdlib.h>
#include <string.h>
#include "arrCarDB.h"
typedef struct {
    char id[6+1];
    char type[9+1];
    char color[9+1];
} Car;

struct CarDB {
    Car DB[10];
    int cars;
};

CDB newCDB(void){
    CDB pDB = malloc(sizeof(struct CarDB));
    pDB->cars=0;
    return pDB;
}
```

arrCarDB.c



OK. Vi övar på det här...

```
#include <stdio.h>
#include "arrCarDB.h"
int main(int argc, char **argv)
{
    CDB myGarage;
    printf("Car DB 1.0 AC\n");

    myGarage = newCDB();

    addCar(myGarage, "WOU069", "MAZDA", "RED");
    addCar(myGarage, "XDW693", "SCODA", "BLUE");
    addCar(myGarage, "WWC099", "SCODA", "GRAY");
    addCar(myGarage, "KLC396", "BMW", "SILVER");
}
```

```
// *** Array based Car DataBase 1.0 AC *****
struct CarDB;
typedef struct CarDB *CDB;
CDB newCDB(void);
void addCar(CDB aDB, char *id, char *type, char *color);
```

arrCarDB.h

```
#include <stdlib.h>
#include <string.h>
#include "arrCarDB.h"
typedef struct {
    char id[6+1];
    char type[9+1];
    char color[9+1];
} Car;

struct CarDB {
    Car DB[10];
    int cars;
};

CDB newCDB(void){
    CDB pDB = malloc(sizeof(struct CarDB));
    pDB->cars=0;
    return pDB;
}

void addCar(CDB aDB, char *id, char *type, char *color){
    // ...
}
```

arrCarDB.c



OK. Vi övar på det här...

```
#include <stdio.h>
#include "arrCarDB.h"
int main(int argc, char **argv)
{
    CDB myGarage;
    printf("Car DB 1.0 AC\n");

    myGarage = newCDB();

    addCar(myGarage, "WOU069", "MAZDA", "RED");
    addCar(myGarage, "XDW693", "SCODA", "BLUE");
    addCar(myGarage, "WWC099", "SCODA", "GRAY");
    addCar(myGarage, "KLC396", "BMW", "SILVER");
}
```

```
// *** Array based Car DataBase 1.0 AC *****
struct CarDB;
typedef struct CarDB *CDB;
CDB newCDB(void);
void addCar(CDB aDB, char *id, char *type, char *color);
```

arrCarDB.h

```
#include <stdlib.h>
#include <string.h>
#include "arrCarDB.h"
typedef struct {
    char id[6+1];
    char type[9+1];
    char color[9+1];
} Car;

struct CarDB {
    Car DB[10];
    int cars;
};

CDB newCDB(void){
    CDB pDB = malloc(sizeof(struct CarDB));
    pDB->cars=0;
    return pDB;
}

void addCar(CDB aDB, char *id, char *type, char *color){
    strcpy(aDB->DB[aDB->cars].id, id);
    strcpy(aDB->DB[aDB->cars].type, type);
    strcpy(aDB->DB[aDB->cars++].color, color);
}
```

arrCarDB.c



OK. Vi övar på det här...

```
#include <stdio.h>
#include "arrCarDB.h"
int main(int argc, char **argv)
{
    CDB myGarage;
    printf("Car DB 1.0 AC\n");

    myGarage = newCDB();

    addCar(myGarage, "WOU069", "MAZDA", "RED");
    addCar(myGarage, "XDW693", "SCODA", "BLUE");
    addCar(myGarage, "WWC099", "SCODA", "GRAY");
    addCar(myGarage, "KLC396", "BMW", "SILVER");

    printf("XXX000 Count = %d\n",
           cntCar(myGarage, 0, "XXX000"));
    printf("MAZDA Count = %d\n",
           cntCar(myGarage, 1, "MAZDA"));
    printf("SCODA Count = %d\n",
           cntCar(myGarage, 1, "SCODA"));
    printf("SILVER Count = %d\n",
           cntCar(myGarage, 2, "SILVER"));

    printf("Car DB Done!\n");
    return 0;
}
```

```
// *** Array based Car DataBase 1.0 AC *****
struct CarDB;
typedef struct CarDB *CDB;
CDB newCDB(void);
void addCar(CDB aDB, char *id, char *type, char *color);
int cntCar(CDB aDB, int member, char *match);
```

arrCarDB.h

```
int id(Car *pCar, char *match){
    return !strcmp(pCar->id, match);
}
int type(Car *pCar, char *match){
    return !strcmp(pCar->type, match);
}
int color(Car *pCar, char *match){
    return !strcmp(pCar->color, match);
}

int cntCar(CDB aDB, int member, char *match){
    int (*marr[])(Car *pCar, char *match) = { id, type, color };
    int cnt=0;

    return cnt;
}
```

arrCarDB.c

- 1) Sända funktion till generiskt paket
- 2) Ersätta switch konstruktioner



OK. Vi övar på det här...

```
Car DB 1.0 AC
XXX000 Count = 0
MAZDA Count = 1
SCODA Count = 2
SILVER Count = 1
Car DB Done!
```

```
#include <stdio.h>
#include "arrCarDB.h"
int main(int argc, char **argv)
{
    CDB myGarage;
    printf("Car DB 1.0 AC\n");

    myGarage = newCDB();

    addCar(myGarage, "WOU069", "MAZDA", "RED");
    addCar(myGarage, "XDW693", "SCODA", "BLUE");
    addCar(myGarage, "WWC099", "SCODA", "GRAY");
    addCar(myGarage, "KLC396", "BMW", "SILVER");

    printf("XXX000 Count = %d\n",
           cntCar(myGarage, 0, "XXX000"));
    printf("MAZDA Count = %d\n",
           cntCar(myGarage, 1, "MAZDA"));
    printf("SCODA Count = %d\n",
           cntCar(myGarage, 1, "SCODA"));
    printf("SILVER Count = %d\n",
           cntCar(myGarage, 2, "SILVER"));

    printf("Car DB Done!\n");
    return 0;
}
```

```
// *** Array based Car DataBase 1.0 AC *****
struct CarDB;
typedef struct CarDB *CDB;
CDB newCDB(void);
void addCar(CDB aDB, char *id, char *type, char *color);
int cntCar(CDB aDB, int member, char *match);
```

arrCarDB.h

```
int id(Car *pCar, char *match){
    return !strcmp(pCar->id, match);
}
int type(Car *pCar, char *match){
    return !strcmp(pCar->type, match);
}
int color(Car *pCar, char *match){
    return !strcmp(pCar->color, match);
}

int cntCar(CDB aDB, int member, char *match){
    int (*marr[])(Car *pCar, char *match) = { id, type, color };
    int cnt=0;
    for (int i=0; i<aDB->cars; i++)
        if ((*marr[member])(&aDB->DB[i], match)) cnt++;
    return cnt;
}
```

arrCarDB.c

- 1) Sända funktion till generiskt paket
- 2) Ersätta switch konstruktioner



Sammanfattning

Incomplete struct, void *, function pointers

malloc() & free()

Explicit memory management:

- The programmer need to explicitly free allocated memory!
- Used in C, C++ and most system level prog. languages.
- Pros: Efficient usage of memory (it is not easy!).
- Cons: Hard to find bugs when you don't do it right.

Implicit memory management:

- Memory is freed by the runtime system.
- Usually a garbage collector (Java, Erlang, Python, .net)
- Pros: Much simpler and/or safer (for the programmer)
- Cons: Could result in runtime overhead and/or lack of control*.

(*Är det en tidskritisk tillämning ljud/bild/styrning av process så är det kanske mindre bra att programmet "stannar" i en sekund och gör garbage collection!)



.h .c #include

**Abstraction,
Reusability &
Maintainability**

**High cohesion
& Low coupling**

**Infor. Hiding:
Security &
Flexibility**

**C incom. types
malloc & free**

void *

**Function
pointers**





Historik

181005 AC 0.1	Started
181008 AC 0.8	Content complete except exercise
181009 AC 0.9	Draft, sent for feedback AL & NB.
181010 AC 1.0	Feedback included, release 1!
200114 AC 1.1	Mindre förtydliganden.
220324 AC 1.2	Mindre förbättringar.